Quorum — Product Document

Final Project · Elevation Academy Version 1.0 · August 9, 2026

1. Team members and roles

MEMBER	ROLE	OWNS
Gal Giladi	Product, Client, Server & Integration Lead	The entire application: React client, Express server, database, debate engine, OpenRouter and Stripe integration, deployment
[Second member — to be confirmed]	Presentation Lead	Demo Day slide deck (PowerPoint) and presenting the product live to the cohort

The second member is not yet confirmed. The build is planned so that it is deliverable by one developer; the presentation is a separable workstream that does not block development.

2. Short product description

Quorum turns a single question into a debate between AI models, then delivers one answer they've argued their way to.

Today, if you want a second opinion on an AI answer, you paste the same question into three different chat apps and compare by hand. Quorum automates that: you assemble a council of models, one of them acts as chairman, and the platform runs a structured four-stage deliberation before returning a single final answer — along with the full record of how it got there.

The point is not that more models are always better. It is that **disagreement between models is a signal**, and today that signal is invisible to users. Quorum surfaces it.

How a round works

STAGE	WHAT HAPPENS	CALLS
1 · Drafts	Every drafting model answers the question independently, in parallel	N−1
2 · Verdict	The chairman receives the drafts **anonymised and shuffled**, then picks one, merges two, or synthesises its own	1
3 · Rebuttals	Each drafter sees the verdict and may **defend, revise, or concede**	N−1
4 · Final	The chairman rules on the rebuttals and produces the final answer	1

Two design decisions worth highlighting:

- **The chairman abstains from drafting by default.** LLMs favour their own output when asked to judge, and anonymising the drafts only partly suppresses it, because models can recognise their own style. Removing the chairman from the drafting pool is the cleaner fix. It stays a user-facing toggle so the effect can be observed.

- **Rebuttals allow concession, not just defence.** If models are only told to defend, they entrench and stage 4 learns nothing. Allowing "I withdraw my point" is what makes the final answer better than the first verdict.

3. Target users

Everyone signs in. There is no anonymous use of the product — the only unauthenticated surface is a read-only shared result page. The meaningful split is between users who have topped up and users who have not.

USER	WHO THEY ARE	WHAT THEY GET
Free user	Registered, wallet empty	**2 debates per day**, full model selection, saved history and presets
Paying user	Registered, wallet funded	Unlimited debates, billed per call against their balance
Admin	Us	Model catalogue and pricing. Post-MVP — see extensions

There is no separate account type. Every user has a wallet; the rules are simply:

- `balance >= max($0.05, estimated_round_cost × 1.5)` → debit the wallet per call.
- Otherwise → allow up to 2 rounds in the current UTC day, then prompt to top up.

The threshold is relative to the round, not a flat floor: a four-model council with an attachment costs many times what a two-model council does, so a fixed minimum would be either too strict or too loose depending on the line-up. The estimate is deliberately worst-case — for each planned call, estimated prompt tokens × input price plus `max_tokens` × output price. Where actual cost overshoots the estimate, the balance is allowed to dip marginally negative and the next round is blocked, rather than building a reservation-and-refund system for fractions of a cent.

This means a paying user whose balance runs out falls back to the free allowance automatically rather than hitting a wall, and topping up is an upgrade with no account migration. **The daily count is a query against `rounds`, not a stored counter** — no reset job, no cron, and it cannot drift out of sync.

Registered users break down into three motivations we design for:

- **The developer** — wants a technical answer cross-checked before acting on it.
- **The analyst / knowledge worker** — high-stakes writing or research where one model's confident error is expensive.
- **The evaluator** — genuinely curious which model is strongest, and wants to watch them argue.

Authentication is email + password *and* Google OAuth, so sign-up friction stays low while we still own the session and authorization layer.

4. Main use cases per target user

Any signed-in user

1. **Register or sign in** with email and password, or with a Google account.
2. **Assemble a council** — toggle models on/off, pick the chairman, set abstain and rebuttal options.
3. **Ask a question**, optionally attaching an image or PDF, and watch the four stages resolve live.
4. **Inspect the reasoning** — expand any individual draft, read the chairman's rubric, see who conceded.
5. **Continue the conversation** — follow-up questions in the same session, with full history.

6. **Change the council mid-session** — swap or add a model; it joins from the next question onward.
7. **Save a council preset** and reuse it later (create, rename, duplicate, delete).
8. **Share a session publicly** — generate a link that anyone can open read-only, without an account.
9. **Review past sessions** — search, filter by verdict type, delete.
10. **See the leaderboard** — a podium of the top three models by win rate, with full standings below.

Leaderboard scoring

Ranking is **win rate over rounds drafted**, never raw wins — otherwise whichever model is toggled on most often wins by default. Rounds in which a model served as chairman are excluded from its denominator, since a judge cannot win its own vote.

VERDICT	SCORING
Chairman picks one draft	1.0 to that model
Chairman merges two drafts	0.5 to each merged model
Chairman synthesises its own answer	No winner; the round still counts as drafted
Model concedes during rebuttal	Recorded separately as concession rate

Two views behind a toggle — **My council** (the signed-in user's sessions) and **All time** (every user) — which is the same aggregate query with an optional `user_id` filter. A model needs at least 5 drafts in the period to be ranked, so a single lucky win cannot top the podium. Everything is computed from `model_responses` and `rounds`; no new tables.

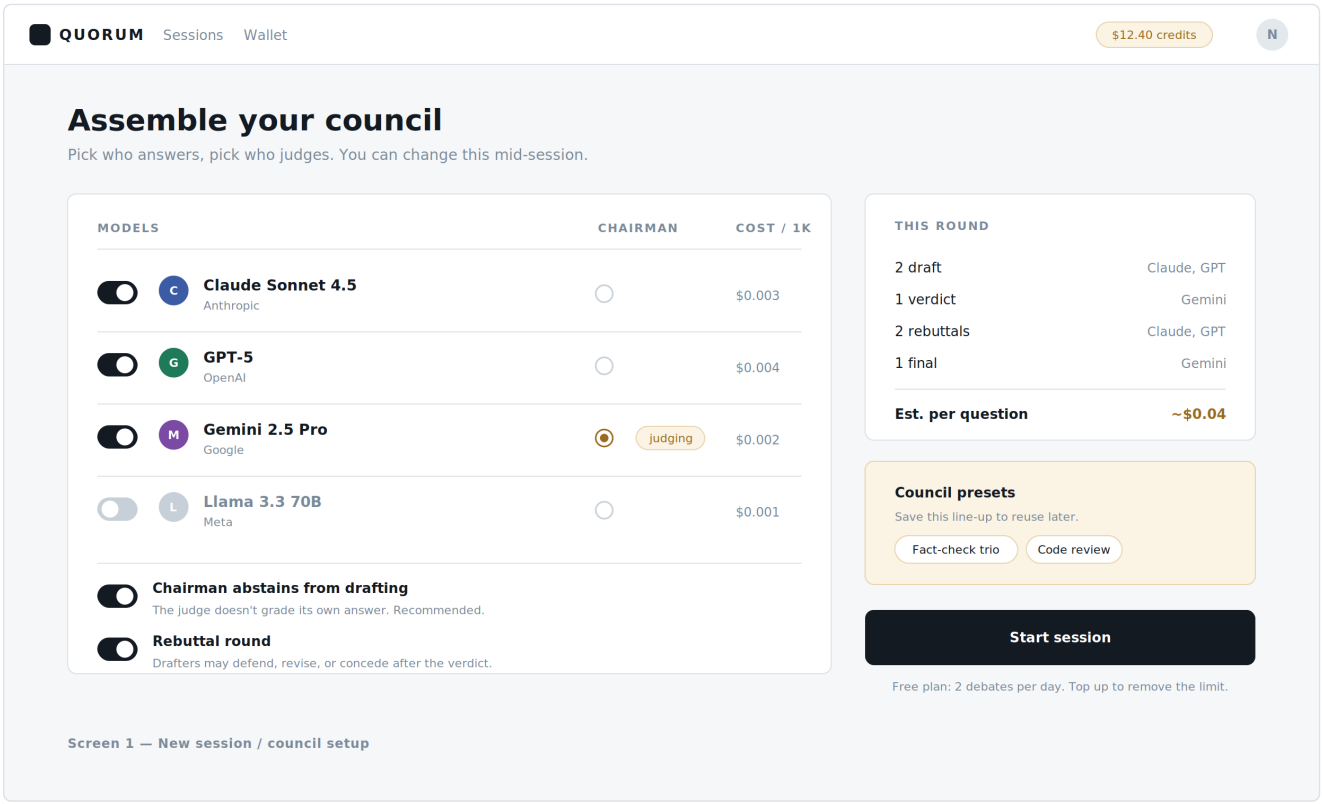
Free user

1. Runs up to **2 debates per day**. The remaining allowance is shown before each send.
2. On hitting the limit, is offered a top-up rather than being blocked from the rest of the app — history, presets, shared links and the leaderboard all stay accessible.

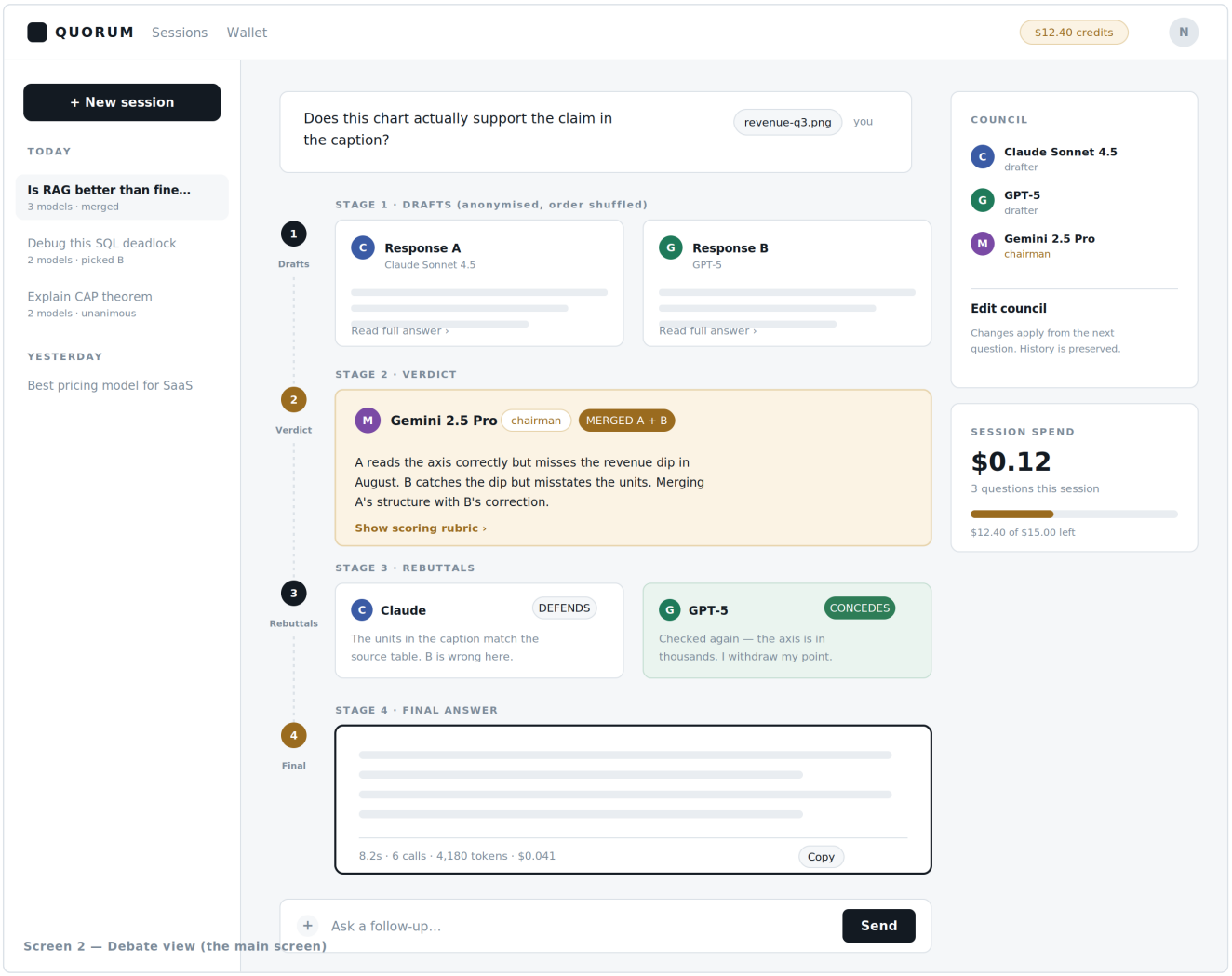
Paying user

1. **Tops up** the wallet via Stripe and debates without a daily cap.
2. **Manages the wallet** — watches the balance fall per call, exports the ledger as CSV.
3. **Sees a pre-flight estimate** of what the next round will cost before sending it.

5. High-level mockups

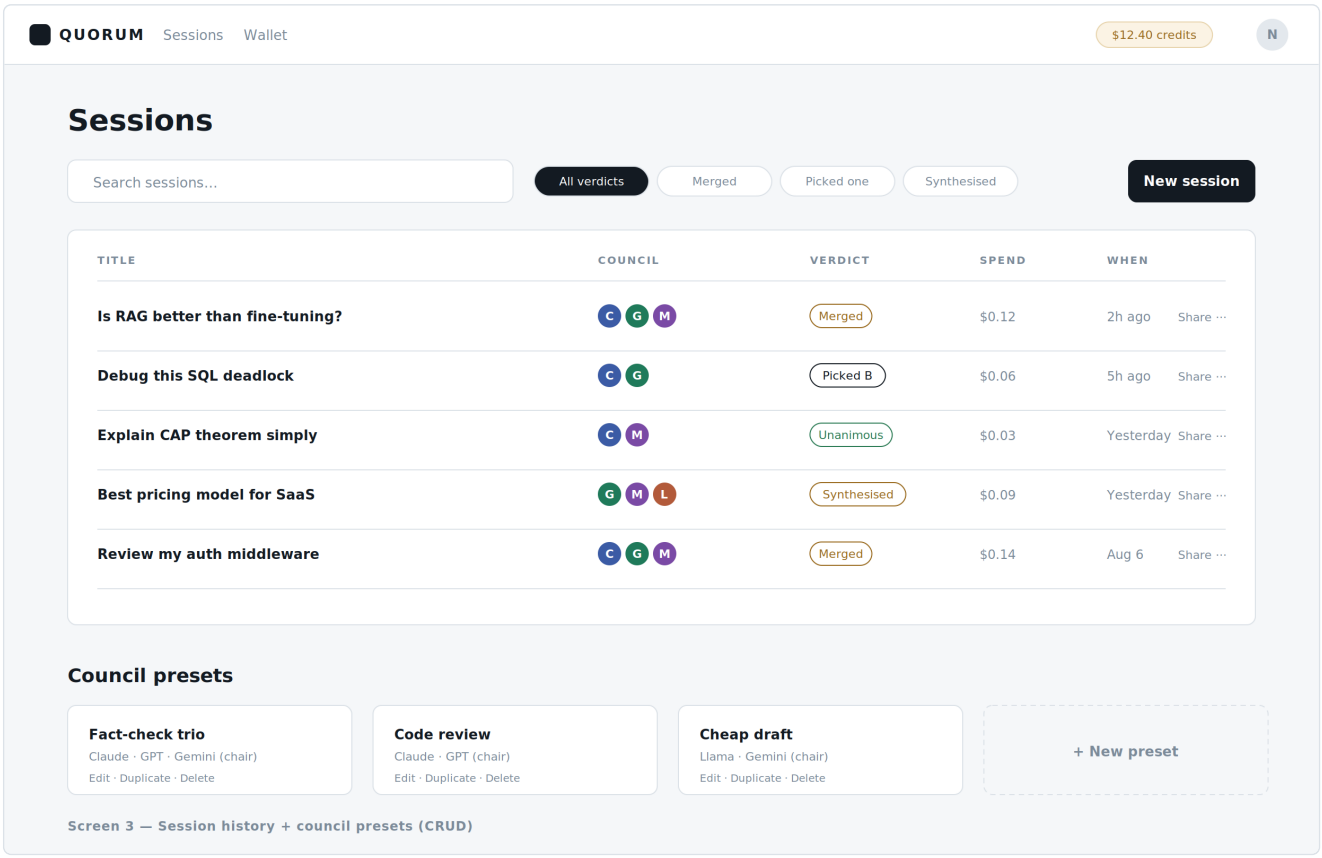


Screen 1 — Council setup: model toggles, chairman selection, abstain and rebuttal switches, per-question cost estimate.

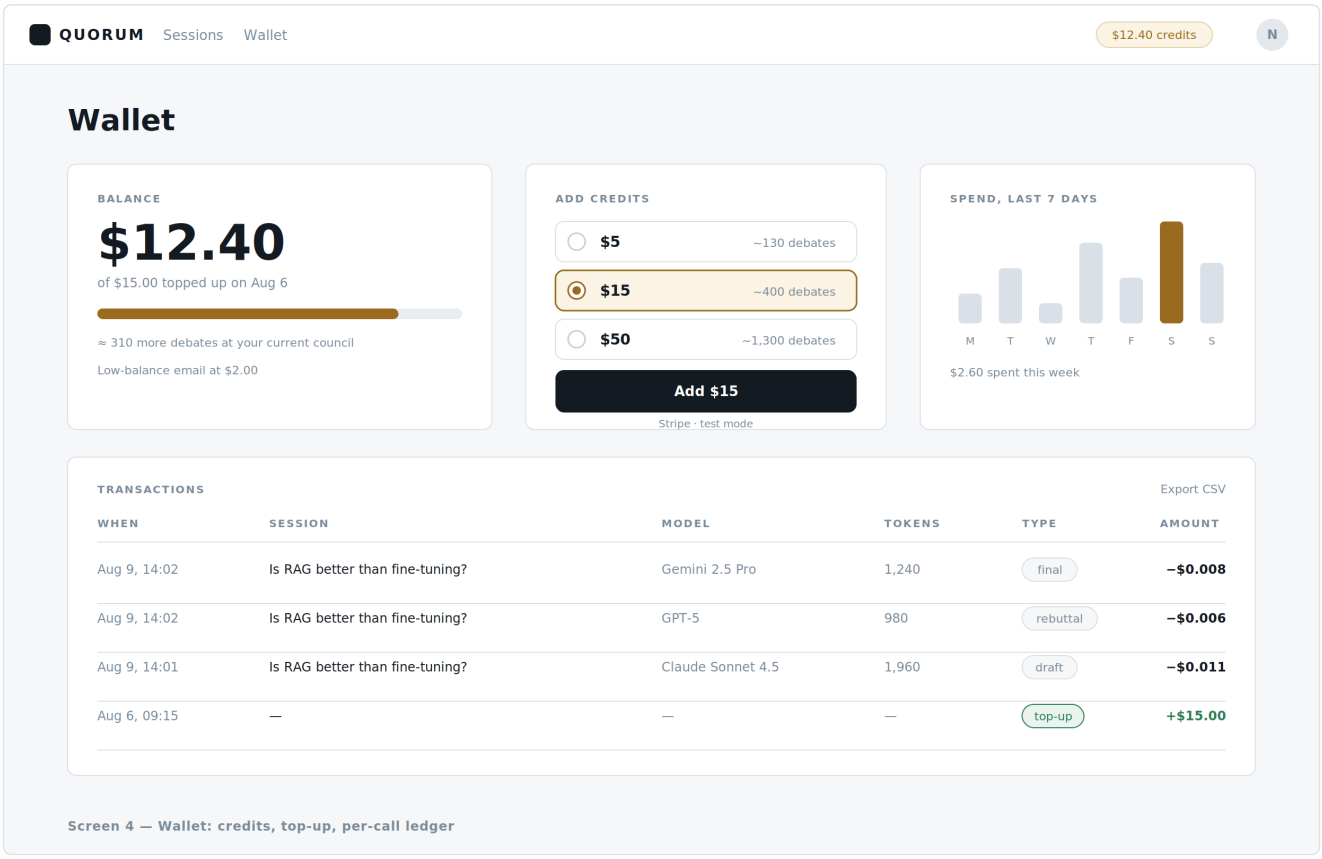


Screen 2 — Debate view (the main screen)

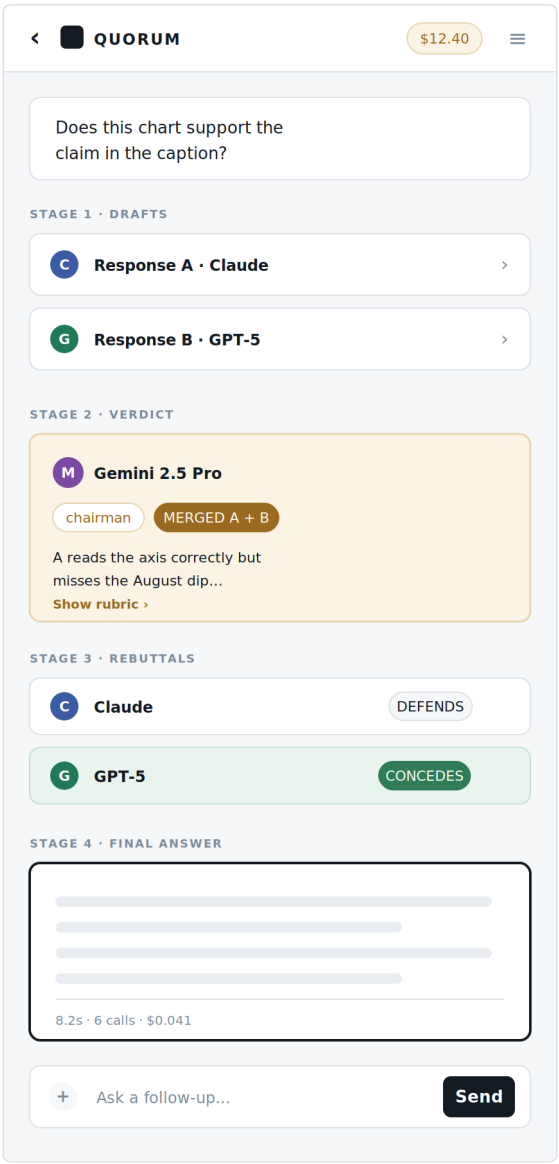
Screen 2 — Debate view. The four-stage rail, anonymised drafts, chairman verdict, rebuttal stances, final answer with cost and timing.



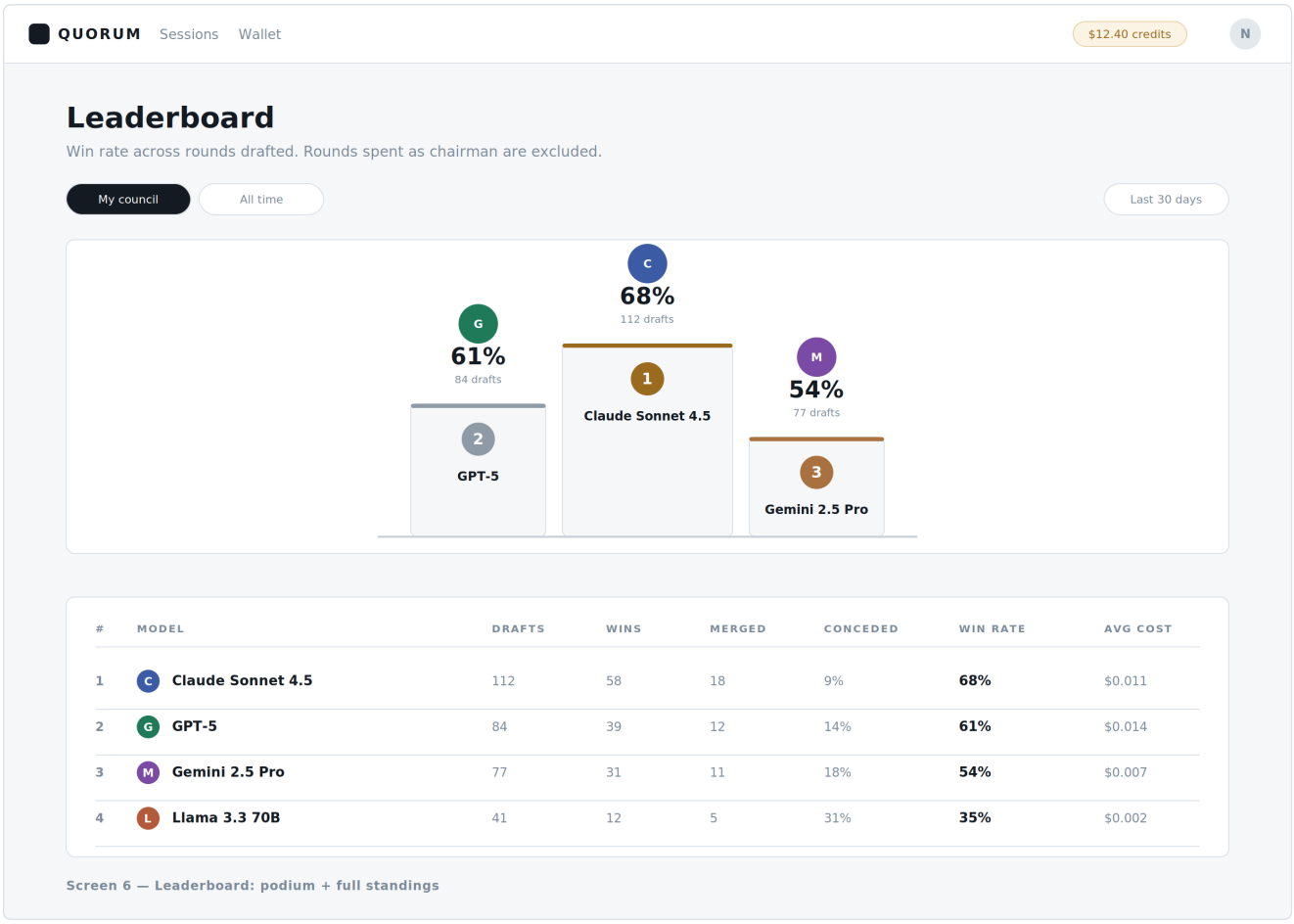
Screen 3 — Session history with verdict filters, plus council preset management.



Screen 4 — Wallet: balance, top-up, weekly spend, per-call transaction ledger.

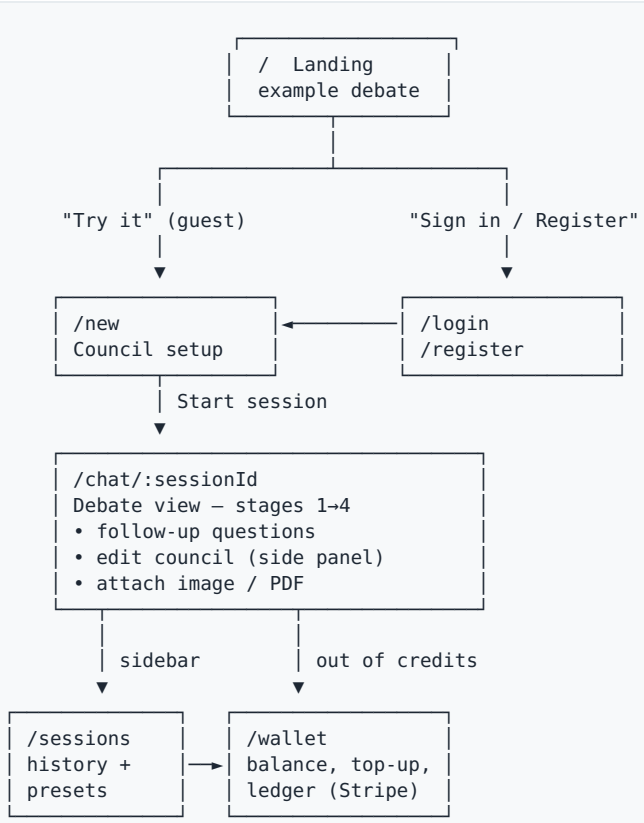


Screen 5 — The debate view on mobile, stages stacked vertically.



Screen 6 — Leaderboard: podium of the top three by win rate, with full standings and cost below.

6. Navigation flow



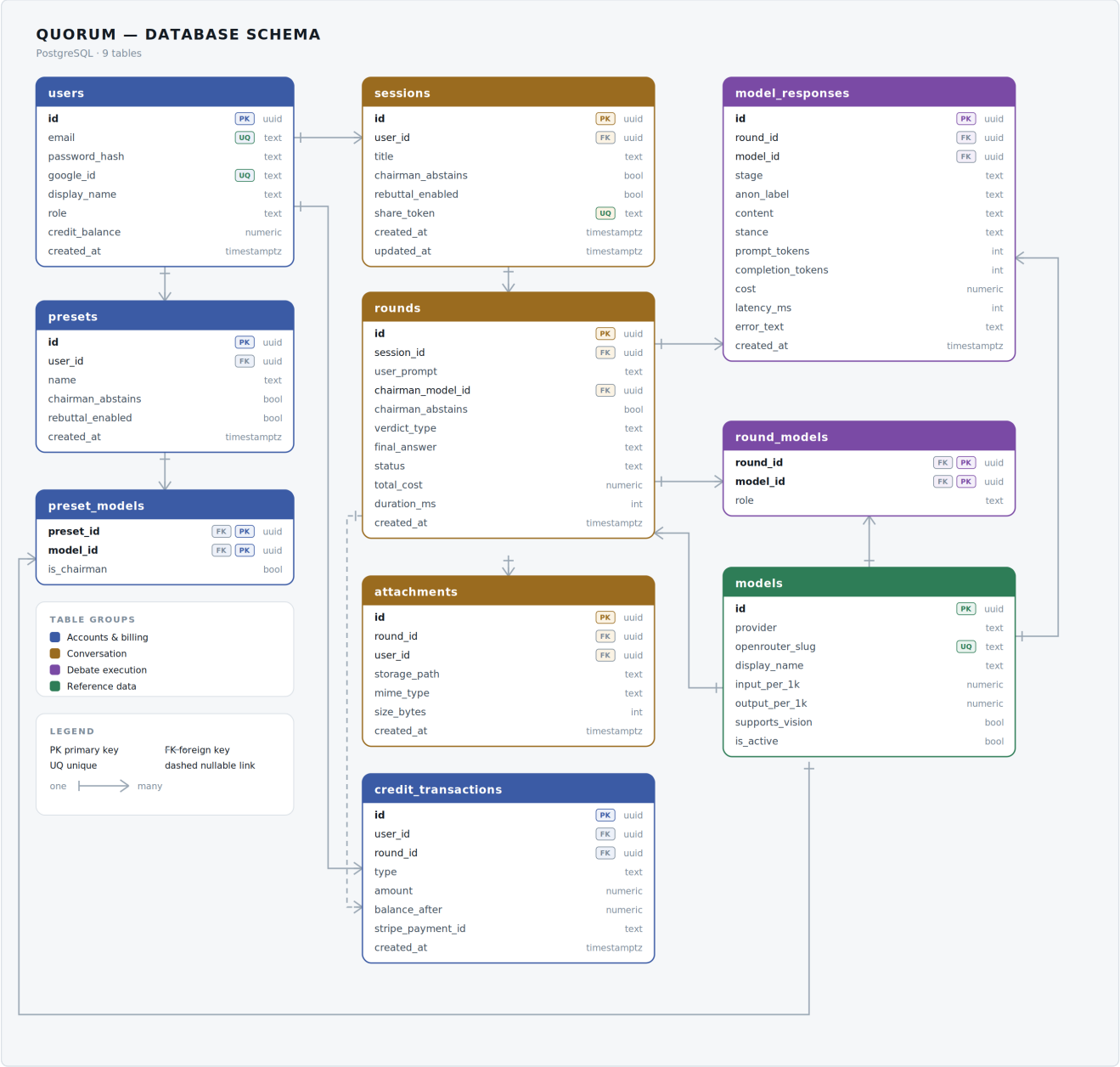
Route table

ROUTE	ACCESS	PURPOSE
`/`	Public	Landing, worked example, CTA
`/login`, `/register`	Public	Email + password, or Google OAuth
`/s/:shareToken`	**Public**	Read-only shared session. The only unauthenticated view
`/new`	Signed in	Council setup
`/chat/:sessionId`	Owner only	Debate view
`/sessions`	Signed in	History + presets
`/leaderboard`	Signed in	Model win, concession and cost statistics
`/wallet`	Signed in	Credits, top-up, ledger

Any protected route hit while signed out redirects to `/login` with the return path preserved. A free user who has used both of the day's debates stays in the app and is offered a top-up rather than a wall.

7. DB diagram (PostgreSQL)

Nine tables in four groups: accounts and billing, conversation, debate execution, and reference data.



Entity-relationship diagram. Bold rows are primary keys; connectors run from the one side to the many side.

Enumerated values

COLUMN	VALUES
`users.role`	`user`, `admin`
`rounds.verdict_type`	`picked`, `merged`, `synthesised`, `unanimous`
`rounds.status`	`drafting`, `verdict`, `rebuttal`, `final`, `complete`, `failed`
`model_responses.stage`	`draft`, `verdict`, `rebuttal`, `final`
`model_responses.stance`	`defend`, `revise`, `concede` (rebuttal stage only)
`credit_transactions.type`	`topup`, `debit`, `refund`, `bonus`

Three schema decisions worth defending in the meeting

1. `round_models` **snapshots the council per round, not per session.** This is what lets a user change models mid-conversation without corrupting history — an old round still shows exactly who participated in it.
2. `model_responses` **stores every single API call**, including failures, with its own token counts and cost. Cost is therefore derived from real usage, never estimated, and the wallet ledger is auditable line by line.
3. `credit_transactions.balance_after` **is stored, not computed.** It makes the ledger immutable and lets us reconstruct any past balance without replaying every row.

8. Server endpoints

Express, MVC (`routes → controllers → services → models`). All responses pass through unified error-handling middleware; all request bodies are validated with Zod.

Auth

METHOD	ENDPOINT	PURPOSE
POST	`/api/auth/register`	Create account, issue JWT (httpOnly cookie)
POST	`/api/auth/login`	Authenticate
GET	`/api/auth/google`	Start Google OAuth
GET	`/api/auth/google/callback`	OAuth callback, issue JWT
POST	`/api/auth/logout`	Clear cookie
GET	`/api/auth/me`	Current user + credit balance

Models & presets

METHOD	ENDPOINT	PURPOSE
GET	`/api/models`	Active model catalogue with pricing
GET	`/api/leaderboard?scope=mine|all&days=30`	Podium and standings: win rate, concession rate, avg cost
GET	`/api/presets`	Current user's presets
POST	`/api/presets`	Create preset
PATCH	`/api/presets/:id`	Rename / change line-up
DELETE	`/api/presets/:id`	Delete preset

Sessions & rounds

METHOD	ENDPOINT	PURPOSE
GET	`/api/sessions`	List (search, filter by verdict)
POST	`/api/sessions`	Create session with a council
GET	`/api/sessions/:id`	Session + all rounds + all responses
PATCH	`/api/sessions/:id`	Rename, or change the council
DELETE	`/api/sessions/:id`	Delete session (cascades)
POST	`/api/sessions/:id/share`	Generate a public share token
DELETE	`/api/sessions/:id/share`	Revoke the share token
GET	`/api/share/:token`	**Public.** Read-only session, no auth
POST	`/api/sessions/:id/rounds`	**Start a debate.** Pre-flight cost check, then run stages 1–4
GET	`/api/rounds/:id/stream`	SSE — emits `stage_started`, `response_ready`, `verdict`, `complete`
GET	`/api/rounds/:id`	Full round detail

Attachments

METHOD	ENDPOINT	PURPOSE
POST	`/api/attachments`	Multipart upload → Supabase Storage, returns signed URL
DELETE	`/api/attachments/:id`	Remove file and row

Wallet

METHOD	ENDPOINT	PURPOSE
GET	`/api/wallet`	Balance, 7-day spend, and today's remaining free debates
GET	`/api/wallet/transactions`	Paginated ledger (CSV export)
POST	`/api/wallet/checkout`	Create Stripe Checkout session
POST	`/api/webhooks/stripe`	Confirm payment, credit the account

9. APIs and services

SERVICE	USED FOR	NOTES
OpenRouter	All LLM calls	One API key and one OpenAI-compatible endpoint reaches Claude, GPT, Gemini and Llama. Usage accounting is automatic: token counts and the actual cost come back in the response body with no extra parameters, and that is what we debit. Calls are non-streaming (`stream: false`) because each stage needs the complete previous output before it can begin. Adding a model becomes a row in `models`, not a new adapter. Hard spend cap set in their dashboard.
Stripe	Credit top-ups	**Test mode.** The integration is production-shaped (Checkout + webhook); only the credentials are test credentials.
Supabase	PostgreSQL + Storage	Postgres for all relational data; Storage as the external media store for prompt attachments.
Vercel / Render	Deployment	React on Vercel, Express on Render.

Authentication is our own: bcrypt password hashing, Google OAuth 2.0 as an alternative sign-in, JWT in an httpOnly cookie, and ownership-checking middleware on every session route. We are not using a hosted auth product, because implementing auth and authorization is a stated requirement of the project.

Streaming to the client uses Server-Sent Events. A round takes 15–25 seconds, so `/api/rounds/:id/stream` pushes an event as each stage completes and the user watches the debate unfold instead of a spinner. SSE rather than WebSockets, because the traffic is one-directional.

Cost control: development runs against cheap small models; flagship models are switched on for demos and screenshots only.

10. Extensions (only if time permits)

EXTENSION	WHY IT'S INTERESTING
Admin panel — model catalogue, pricing, failed-round inspection, refunds	Useful operationally, but nothing in the demo depends on it; model rows can be managed directly in SQL until then
BYOK — users supply their own provider keys, encrypted at rest	Removes our cost exposure entirely; demonstrates key management
Self-preference measurement — run the same prompt with the chairman drafting and abstaining, and chart the win-rate difference	Turns the project into a small research result rather than only an integration
Token-by-token streaming on the final answer (`stream: true` on the last call only, relayed through the existing SSE channel)	Purely cosmetic, but it makes the final answer feel alive
Multi-round debate — more than one rebuttal cycle, with an automatic stop when models converge	Deeper deliberation; needs careful termination logic
Team councils — shared presets across a group of users	Natural next step once sharing exists

11. Known risks

RISK	MITIGATION
A four-stage round is slow (~15–25s)	Stages 1 and 3 run in parallel via `Promise.allSettled`; results stream to the UI stage by stage, so the user always sees progress
One provider times out or errors	`allSettled`, not `all` — the round continues with the models that answered, and the failure is recorded in `model_responses.error_text`
Runaway API spend	Hard cap at OpenRouter, per-user credit balance, pre-flight cost check before stage 1
Scope is large for one developer in 7 days	Debate engine and wallet are the core; presets, admin panel and CSV export are cut first if we slip
Second member not yet confirmed	The build plan assumes a single developer. If a partner joins, the Wallet and Stripe workstream is the cleanest slice to hand over
Free tier abused by repeat sign-ups	The daily allowance is small and per account; Google OAuth raises the cost of creating throwaway accounts. Not worth further defence at this scale
Public share links leak private content	Sharing is opt-in per session, the token is random and revocable, and the shared view excludes wallet and account data

12. Build plan

DATES	OWNER	MILESTONE
Aug 9–10	Gal	Schema and migrations, auth, model catalogue, OpenRouter service, single-model round working end to end
Aug 11–13	Gal	Full four-stage debate engine, SSE streaming, debate view UI
Aug 14–15	Gal	Wallet, Stripe test checkout, ledger, free-tier daily limit, pre-flight cost check
Aug 16	Gal	Presets, session history, share links, leaderboard, attachments, mobile responsiveness. **Application complete.**
Aug 17–18	Partner	Slide deck: product overview, problem statement, technical highlights
Aug 17–20	Gal	Bug fixes, deployment hardening, screen recordings for the deck
Aug 19	Both	Feature close (bootcamp deadline). Deck reviewed against the live app
Aug 20	Both	Demo rehearsal, 7-minute run-through, Q&A preparation

Building to Aug 16 rather than the Aug 19 feature-close deadline leaves three days of deliberate buffer. Nothing in the deck can be finalised until the screens it shows actually exist, so the application finishing early is what makes the presentation workstream possible.